READING PREVIEW — not the submission file. The camera-ready PDF must be produced by compiling `finguard.tex` with the IEEEtran class (e.g. on Overleaf). This preview exists so the content can be reviewed without a LaTeX toolchain; layout, two-column formatting, and page count will differ. Red markers show placeholders still to be filled.

RT-IIPF: Real-Time Inline Interception of Fraudulent UPI Transactions with Transactional Rollback and Unsupervised Anomaly Scoring

Author block contains unfilled placeholders — see red markers

Abstract

Conventional payment-fraud analytics operate retrospectively: transactions settle, logs are analysed downstream, and recovery is attempted after funds have already moved. In high-velocity real-time rails such as India's Unified Payments Interface (UPI), this latency renders recovery frequently impossible. We present RT-IIPF, a working prototype that relocates the fraud decision *inside* the payment transaction boundary. The switch acquires pessimistic locks on both ledger rows, applies the debit and credit provisionally within an open ACID transaction, suspends the executing thread pending a verdict from an isolated inference microservice, and then either commits or aborts. A fraudulent transfer is rolled back atomically and never attains durable existence, while the account lock and audit record survive the rollback via an independent transaction. We evaluate the system on 250,000 UPI transactions. Because the public dataset's fraud labels are statistically uninformative (ROC-AUC ≈0.48 for every detector tested), we adopt perturbation-based synthetic fraud injection with an explicit negative control. Three findings are reported, including negative ones: (i) an ablation across enrichment layers shows that client telemetry raises operating-point recall 22× while *lowering* ROC-AUC, indicating that AUC is the wrong selection criterion under a false-positive budget; (ii) Local Outlier Factor outperforms the deployed Isolation Forest (recall 0.745 vs. 0.163 at equal 1% FPR), showing the architectural contribution to be orthogonal to and currently bottlenecked by model choice; and (iii) concurrency measurements falsify the expected hypothesis that lock-holding across the scoring hop limits throughput — the constraint instead resides in the stateless inference tier, where four-way replication yields 2.0–3.1× speed-up. Single-client end-to-end latency is 22.3\,ms.

Index terms—fraud detection, anomaly detection, isolation forest, UPI, real-time payments, microservices, ACID transactions, unsupervised learning

Introduction

The Unified Payments Interface processes billions of transactions monthly, settling each in seconds. This velocity, which underpins UPI's utility, also compresses the window available for fraud intervention to near zero. Once a fraudulent transfer settles, funds are typically forwarded through intermediary accounts and withdrawn before any retrospective analytic pipeline raises

an alert. The economics are asymmetric: detection accuracy achieved after settlement contributes little if recovery is infeasible.

The dominant architectural pattern in deployed fraud analytics remains post-facto: transaction logs are consumed by batch or streaming pipelines that score transactions after they have committed, and flagged cases enter a manual review or recovery workflow. We argue that for real-time rails the decision must be relocated *into* the transaction path, such that a rejected payment is never durably recorded.

This paper describes RT-IIPF (Real-Time Inline Interception Payment Framework), a functioning three-service prototype implementing this relocation, together with an evaluation that is deliberate about reporting negative results. Our contributions are:

1. **An inline interception mechanism** in which the fraud verdict arbitrates an open database transaction, with a two-transaction pattern that allows the security response (account lock, audit record) to survive the ledger rollback it describes (Section *arch*).

\item **A layered feature-enrichment design with an explicit trust boundary**, in which client-supplied features are limited to those where adversarial forgery increases rather than decreases anomaly, and decisive temporal and demographic features are server-resolved (Section *enrich*).

\item **An evaluation methodology for unsupervised fraud scoring under unreliable public labels**, using perturbation-based synthetic injection with a negative control that quantifies an architectural blind spot rather than concealing it (Section *method*).

\item **Three measured findings, two of them negative**: the inadequacy of ROC-AUC as a selection metric under false-positive budgets; the superiority of a density-based baseline over the deployed detector; and the empirical falsification of our own expected scaling bottleneck (Section *results*).

We explicitly do not claim that real-time transaction scoring is itself novel; variants are known in industrial practice. Our contribution is an open, reproducible, end-to-end prototype exhibiting the transactional mechanics in full, accompanied by an honest characterisation of where such a design succeeds and where it does not.

Related Work

Isolation Forest [liu2008] detects anomalies by exploiting the observation that outliers require fewer random axis-parallel partitions to isolate than inliers, yielding linear time complexity and low memory footprint — properties attractive for latency-constrained inference. Local Outlier Factor [breunig2000] instead assigns a degree of outlierness by comparing local reachability densities, and One-Class SVM [scholkopf2001] estimates a support region enclosing the normal class. Comparative studies of unsupervised outlier detectors [campos2016] report substantial task-dependent variation in relative ranking, motivating empirical baseline comparison rather than a priori model selection.

Surveys of financial fraud detection [abdallah2016,adewumi2017] document the field's predominant reliance on supervised learning over labelled historical fraud, and note the recurring obstacles of extreme class imbalance, label latency, and concept drift. These obstacles

motivate unsupervised formulations, which require no fraud labels and can in principle respond to novel attack patterns absent from historical data.

TODO: Add 2–4 citations specific to UPI or Indian real-time payment security, and one on transactional/ACID guarantees in payment switches, to strengthen venue fit.

System Architecture

Service decomposition

RT-IIPF comprises three processes communicating over HTTP/JSON: a React client presenting the payment interface and an operational monitor; a Spring Boot 3 switch (Java 21) owning authentication, the ledger, and transactional control; and a FastAPI inference node (Python 3) exposing a single scoring endpoint. The ledger is an embedded H2 relational database accessed through Spring Data JPA, comprising `users`, `balances`, and `transaction_logs`.

The interception pipeline

On receipt of a payment request the switch executes the following within one `@Transactional` method:

1. Resolve payer and payee by virtual payment address; reject locked accounts; verify the MPIN digest in constant time.
2. Acquire `PESSIMISTIC_WRITE` locks on both balance rows, always in ascending account-identifier order so that opposing concurrent payments cannot deadlock.
3. Apply the debit and credit to the persistence context. These changes are real but uncommitted — the transaction exists provisionally.
4. Issue a synchronous HTTP request to the inference node carrying the assembled feature vector, blocking the request thread. Connect and read timeouts are bounded (300\,ms, 1500\,ms) because ledger locks are held for the duration.
5. Branch on the returned verdict.

Rollback with surviving audit

A verdict of fraud creates an apparent contradiction: the ledger movement must be erased, yet the institution must durably record the attempt and immobilise the account. Naively writing both inside the payment transaction would destroy the audit record along with the movement.

RT-IIPF resolves this with a two-transaction pattern. The account lock and the `BLOCKED_FRAUD` audit row are written by a component annotated `Propagation.REQUIRES_NEW`, which suspends the payment transaction, commits independently, and returns. Only then does the payment path throw `FraudulentTransactionException`; the framework marks the suspended transaction rollback-only and the provisional debit and credit are discarded atomically. The commit path is simply the absence of this exception. The observable postcondition of a blocked payment is therefore: balances unchanged, account locked, audit row present.

Fail-closed availability posture

If the inference node is unreachable or exceeds its timeout, the switch rejects the payment (HTTP 503) rather than admitting it unscored. Safety is preferred to availability; Section *limits*

discusses the cost.

Feature Enrichment and Trust Boundary

The client interface collects six fields, of which three reach the model (amount, transaction type, merchant category); the two addresses serve as routing keys and the MPIN as an authentication secret that never leaves the switch. The model receives thirteen features, assembled in three layers:

- **L1 — user-entered** (3 features): amount, transaction type, merchant category.
- **L2 — client telemetry** (2 features): device class and network type, captured autonomously by the client at page load without user interaction.
- **L3 — server enrichment** (8 features): counterparty age bands, sender state, both issuing banks, resolved from the account database; and hour, weekday, and weekend flag, resolved from the switch's own clock.

Enrichment imposes negligible latency because both account rows are already materialised for authentication and routing; no additional query is issued.

The partition is a deliberate trust boundary. Features an adversary controls are those for which anomalous values *increase* the anomaly score, so misreporting is self-defeating. Features that would most benefit an adversary — transacting at a plausible hour, presenting a normal demographic profile — are resolved server-side and are not client-assertable.

Evaluation Methodology

Motivation for synthetic injection

The evaluation corpus comprises 250,000 UPI transactions with seventeen attributes, split 80/20 into training and held-out sets. The corpus carries a `fraud_flag` annotation, which we found to be statistically uninformative: no detector evaluated in this work, supervised or unsupervised, exceeded ROC-AUC 0.48 against it, a value consistent with random assignment. Reporting accuracy against such labels would be vacuous.

We therefore adopt perturbation-based synthetic injection. Four fraud archetypes are instantiated by sampling genuine held-out transactions and overwriting only the attributes that define each archetype, leaving all remaining attributes at realistic sampled values so that synthetics are not trivially separable by marginal inspection. One hundred vectors per archetype (400 total) are injected into 50,000 clean held-out transactions, yielding 0.79% synthetic prevalence.

Archetypes and the negative control

- **A — off-hour high-value drain**: INR 25,000–60,000, 01:00–04:00, peer-to-peer, web origin.
- **B — velocity spike (negative control)**: INR 2,000–6,000, business hours, common device and network. Each constituent transfer is individually unremarkable; the attack exists only in aggregate.

- **C — category anomaly**: INR 50,000–100,000 under an everyday merchant category, daytime.
- **D — context anomaly**: INR 2,000–9,000, 00:00–04:00, web and Wi-Fi origin.

Archetype B is included as a negative control. The deployed model scores each transaction independently and receives no velocity features; it is therefore *expected* to detect none of these vectors. Including B quantifies a known architectural boundary instead of selecting archetypes favourable to the system.

Experimental configuration

All experiments use seed 42 and are reproducible from the released evaluation script. The deployed configuration is an Isolation Forest of 200 estimators with contamination 0.002; raw scores are mapped to [0,1] by min–max normalisation whose upper bound is winsorised at the 99.95th percentile of the training score distribution, preventing a single extremal training point from compressing the operational range. The production threshold τ=0.85 was chosen to bound empirical false-positive rate below 1%. Score-based baselines are compared at an equal 1% false-positive budget, reflecting the operational reality that an inline blocker is FPR-constrained rather than accuracy-constrained.

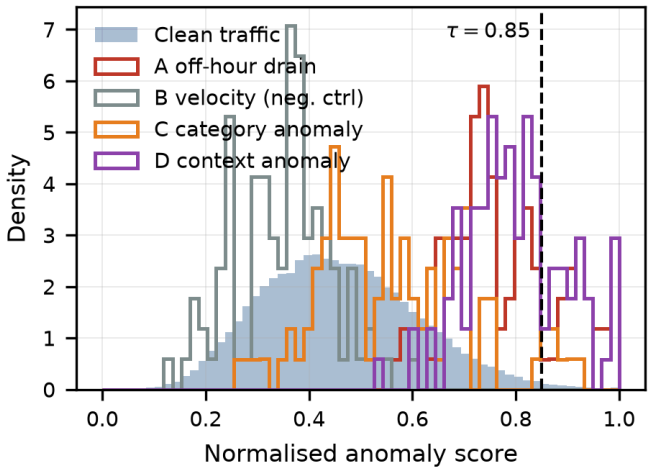
Results

Detection under injection

Detection by archetype at the production operating point (τ=0.85), n=100 per archetype

	Archetype	Mean score	Recall
A	Off-hour high-value drain	0.781	0.26
B	Velocity spike *(neg. control)*	0.350	0.00
C	Category anomaly	0.567	0.07
D	Context anomaly	0.799	0.26
Clean held-out traffic (n=50,000)}	FPR =0.0086		

Table *arch* reports detection by archetype. Overall precision is 0.121, recall 0.148, F_1 0.133, at ROC-AUC 0.723 and a false-positive rate of 0.86%.



Score distributions of clean traffic and each injected archetype. Archetypes A and D concentrate immediately *below* the production threshold, visualising isolation saturation; the velocity negative control (B) lies inside the clean distribution, confirming that a per-transaction scorer cannot separate it.

Two failure mechanisms are identifiable, both visible in Fig. *dists*. First, *isolation saturation*: amounts beyond the training maximum cannot deepen an isolation path further, so archetypes defined principally by extremal amount (A, C) asymptote below τ despite being conspicuous to inspection — their mass accumulates immediately below the threshold rather than crossing it. Second, the velocity blind spot is total, exactly as the negative control predicted: distribution B is indistinguishable from clean traffic.

Ablation of enrichment layers

Enrichment-layer ablation; each layer independently trained and calibrated

Layer	Feat.	ROC-AUC	Recall@τ	Recall D
L1 form only	3	**0.860**	0.010	0.00
L2 + telemetry	5	0.747	**0.228**	**0.40**
L3 + server enr.	13	0.723	0.148	0.26

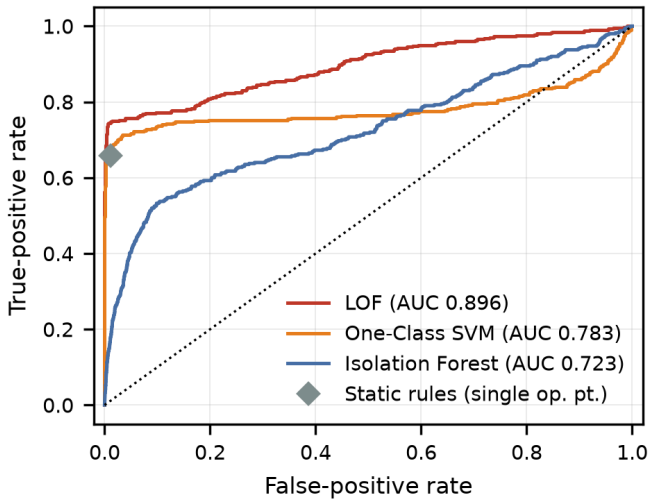
Table *ablation* exhibits a dissociation between ranking quality and operating-point utility. The form-only model attains the *highest* ROC-AUC (0.860), its ranking dominated by the amount marginal, yet it is operationally inert: it detects 1% of attacks at the deployable threshold and none of archetype D. Adding client telemetry sacrifices global AUC (0.747) while multiplying operating-point recall twenty-two-fold (0.010 rightarrow 0.228) and enabling context-anomaly detection entirely absent at L1 (0.00 rightarrow 0.40).

Two conclusions follow. Enrichment features determine *which attack classes are detectable at all* under a fixed false-positive budget; and ROC-AUC, which integrates over operating points that an inline blocker can never adopt, is a misleading model-selection criterion for this task. Recall at the deployed threshold is the decision-relevant metric.

Baseline comparison

Baselines at an equal 1% false-positive budget

Detector	AUC	Recall	Prec.	F_1
Static rules	—	0.658	0.305	0.417
Local Outlier Factor	**0.896**	**0.745**	**0.373**	**0.497**
One-Class SVM	0.783	0.675	0.351	0.462
Isolation Forest (depl.)	0.723	0.163	—	—



ROC comparison on the injected set. The rule baseline emits a binary decision rather than a ranking and is therefore shown as its single operating point; deriving a continuous pseudo-score for it would be amount-dominated and hence near-oracular for the amount-defined archetypes, an artefact of archetype construction rather than a property of the classifier.

Table *baselines* and Fig. *roc* report a negative result for the deployed model. Local Outlier Factor substantially outperforms Isolation Forest (recall 0.745 vs. 0.163), as do One-Class SVM and even static amount-and-hour rules. We attribute the deficit to Isolation Forest's known degradation on sparse one-hot encoded categorical spaces, where axis-parallel splits dilute the informative numeric dimensions, compounded by the saturation effect of Section VI-A.

We report this deliberately. It establishes that the architectural contribution of RT-IIPF — inline interception with transactional rollback — is orthogonal to, and presently bottlenecked by, the choice of scoring model. The interception mechanism is model-agnostic; substituting a density-based scorer is an immediate and well-characterised uplift path.

Latency and concurrency

End-to-end throughput and latency; 200 requests per level

	Uncontended		Contended	
Conc.	**TPS**	**p95 (ms)**	**TPS**	**p95 (ms)**
1	44.5	23.8	47.2	21.9
10	49.2	63.2	50.4	61.7
25	49.5	374.5	50.6	372.5
50	50.0	864.2	50.8	849.1

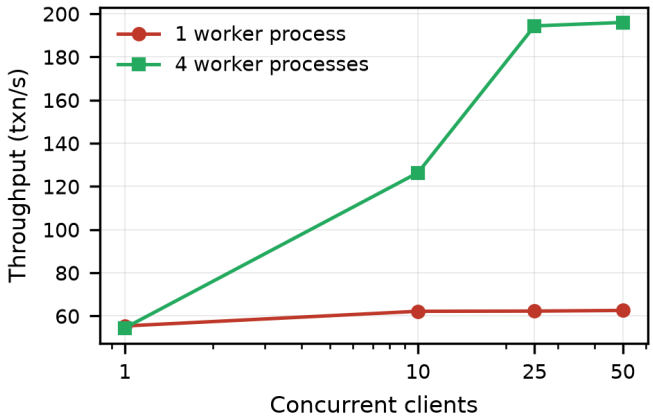
Scoring node in isolation: single process vs. four workers

	1 worker		4 workers	
Conc.	**TPS**	**p95**	**TPS**	**p95**
1	55.4	18.5	54.5	19.0
10	62.2	205.9	**126.4**	125.9
25	62.3	522.0	**194.4**	217.9
50	62.6	924.2	**196.0**	382.5

Under a single client the full inline path completes in 22.3\,ms mean (mathrmp50 22.0\,ms, mathrmp99 29.2\,ms), inside the 50\,ms design budget. Decomposing this figure, the hybrid scoring hop accounts for 18.7\,ms and switch-side work — authentication, lock acquisition, persistence and commit — for only 3.6\,ms. Within the scoring hop the Isolation Forest tier costs 9.25\,ms, preprocessing 3.56\,ms, the LOF tier 3.08\,ms and the rule tier 0.11\,ms. The forest is thus simultaneously the most expensive tier and, by Table *baselines*, the least discriminative.

Under concurrent load (Table *conc*) throughput saturates near 50\,TPS while latency grows linearly with offered concurrency — the signature of one saturated serial resource. Critically, contended and uncontended regimes are statistically indistinguishable (50.8 vs. 50.0 TPS at concurrency 50). This *falsifies* the hypothesis we set out to test, namely that pessimistic ledger locks held across the scoring round-trip constitute the scaling constraint.

Isolating the scoring node (Table *scale*) locates the true bottleneck. Served by a single Python process, `/score` throughput plateaus near 62\,TPS while p95 latency rises to 924\,ms at concurrency 50, consistent with global-interpreter-lock contention among threads executing CPU-bound vectorised inference. Because the node is stateless, the remedy is horizontal: four worker processes yield 126.4\,TPS at concurrency 10 and 196.0\,TPS at concurrency 50 — speed-ups of 2.0× and 3.1× — while reducing p95 latency at concurrency 50 from 924\,ms to 383\,ms.



Scoring-node throughput. A single process plateaus near 62\,TPS under rising concurrency (GIL contention), with p95 latency growing to 924\,ms; four worker processes raise capacity to 196\,TPS, confirming the bottleneck lies in the stateless tier rather than in the transactional ledger path.

This is architecturally consequential. The design decision most exposed to criticism — holding row locks across a synchronous network call — is empirically not limiting at the concurrency levels examined, whereas the limit resides in the one tier that is stateless and therefore trivially replicable (Fig. *scaling*). Interception correctness and scoring capacity scale independently.

Limitations and Future Work

Archetype-design bias. Three of the four archetypes (A, C, and to a lesser extent D) manipulate transaction amount, so the injected set is partially amount-separable by construction. This favours amount-sensitive scorers in *ranking* metrics — an effect we observed directly, and which underlies the anomalously high AUC of any amount-dominated score on this set. It does not affect the operating-point comparisons, which are budget-matched, but a fuller evaluation would include archetypes defined solely by categorical or temporal deviation with amounts drawn from the clean marginal.

Population-level modelling. The detector encodes what is anomalous for the population, not for the individual account. A customer routinely transferring large sums is indistinguishable from a compromised account draining for the first time. Per-account rolling baselines — deviation from historical amount distribution, novel-payee and novel-device indicators — are the highest-value feature additions and are computable from the existing audit table.

Velocity blindness (measured). The negative control quantifies a hard boundary: zero detection of rapid-drain sequences. Windowed velocity features over one-minute and one-hour horizons require a single indexed query at enrichment time and a model retrain.

Model selection. The baseline study identifies LOF as dominant at equal FPR under this encoding. A two-tier arrangement — inexpensive forest prefilter, density-based confirmation — and target encoding of categoricals to relieve one-hot dilution are the indicated next steps.

Graduated friction. The prototype hard-blocks and locks at tau geq 0.85. At the measured 0.86% false-positive rate, such a posture applied at national scale would immobilise a substantial number of legitimate accounts daily. A production design would introduce step-up authentication in an intermediate band, reserve hard blocks for extreme scores, and release locks through a support workflow rather than permanently.

Fail-closed availability. Scoring unavailability currently rejects all payments. A degraded-mode local rule engine behind circuit-breaker semantics would bound the availability cost of the safety preference.

Telemetry trust boundary. Client-originated features remain spoofable. The design mitigates but does not eliminate this, since plausible spoofing shifts a transaction toward rarer, higher-scoring contexts. Server-side verification of IP-derived geolocation would strengthen the boundary.

Prototype security scope. MPIN digests use unsalted SHA-256 rather than a memory-hard key-derivation function; transport is unencrypted; there is no session authentication. These were excluded deliberately to isolate latency measurement and are prerequisites for deployment.

Measurement scope. All figures are single-host and loopback-networked, excluding datacenter latency, TLS termination, and multi-tenant interference. Isolated approx4\,s p99 outliers at burst onset, attributed to JVM JIT warm-up and non-pooled harness connections, are reported unfiltered and warrant elimination before publication-grade tail-latency claims.

Conclusion

We presented RT-IIPF, a working prototype relocating the payment-fraud decision inside the transaction boundary, such that a rejected transfer is rolled back atomically and never durably exists, while the security response survives that rollback via an independent transaction. Evaluation under synthetic fraud injection — adopted because the public corpus's labels proved uninformative — yielded three findings we report irrespective of their favourability: that ROC-AUC misleads as a selection criterion under false-positive budgets, that a density-based baseline outperforms the deployed detector, and that our own expected scaling bottleneck was falsified by measurement, the true constraint lying in a stateless and therefore replicable tier. The interception architecture is sound and model-agnostic; its detection quality is presently limited by scorer choice and by the absence of per-account and velocity features, both of which the existing data model can support.

Reproducibility

Source, the evaluation harness, and the concurrency benchmark are available at **TODO: repository URL**. All experiments use seed 42.

References

1. F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, Dec. 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.
2. M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: Identifying density-based local outliers,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Dallas, TX, USA, May 2000, pp. 93–104, doi: 10.1145/342009.335388.
3. B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, “Estimating the support of a high-dimensional distribution,” *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001, doi: 10.1162/089976601750264965.

4. G. O. Campos *et al.*, “On the evaluation of unsupervised outlier detection: measures, datasets, and an empirical study,” *Data Mining and Knowledge Discovery*, vol. 30, no. 4, pp. 891–927, 2016, doi: 10.1007/s10618-015-0444-8.

5. A. Abdallah, M. A. Maarof, and A. Zainal, “Fraud detection system: A survey,” *Journal of Network and Computer Applications*, vol. 68, pp. 90–113, 2016, doi: 10.1016/j.jnca.2016.04.007.

6. A. O. Adewumi and A. A. Akinyelu, “A survey of machine-learning and nature-inspired based credit card fraud detection techniques,” *Int. J. System Assurance Engineering and Management*, vol. 8, no. 2, pp. 937–953, 2017, doi: 10.1007/s13198-016-0551-y. **TODO: Add 2–4 UPI-specific or Indian real-time-payment security references to strengthen venue fit.** \end{thebibliography}